

- Revised Architecture for PDFBooker

1. Frontend Components

- URL Input Form: For entering target URLs and crawl depth
- Configuration Panel: Options for content filters, output format, templates
- Authentication UI: Login/signup with username/password, Google, Github OAuth
- User Dashboard: View and manage previously created PDFs
- Template Gallery: Pre-designed PDF templates with preview
- Customization Panel: Font, color, layout customization
- Progress Tracker: Real-time progress indicators for each step
- PDF Preview: Interactive preview before download
- Job History: List of previous and current jobs

2. Backend Architecture

Backend will use Next.js API routes with a job queue system:

- Authentication Service: Handle user auth, sessions, and permissions
- Job Orchestrator: Manage the lifecycle of scraping jobs
- Queue System: Process jobs asynchronously (using Bull or similar)
- Rate Limiter: Prevent abuse and manage crawling frequency
- Compliance Manager: Handle robots.txt parsing and rules
- Storage Service: Manage temporary and persistent storage
- API Gateway: Expose endpoints for frontend and external access

3. Content Processing Pipeline

Multi-stage pipeline for processing web content:

- URL Validator: Validate and normalize input URLs
- Crawler Service: Recursively crawl pages respecting depth limits
- Content Extractor: Extract main content using readability algorithms
- Media Processor: Handle images and other media elements
- Link Resolver: Resolve relative links for proper navigation
- Content Cleaner: Remove ads, nav bars, and non-essential elements
- Content Organizer: Structure content in logical reading order
- Mobile Detector: Optimize for mobile content when available
- Error Handler: Manage and report processing errors

4. PDF Generation Module

Responsible for creating the final PDF output:

- Document Assembler: Combine processed content into PDF structure
- TOC Generator: Create hierarchical table of contents
- Cover Generator: Design and add cover pages
- Styling Engine: Apply templates and custom styling
- Bookmark Creator: Add PDF bookmarks for navigation
- Metadata Manager: Add and manage PDF metadata
- Layout Engine: Handle responsive layouts for different page sizes
- Image Optimizer: Balance quality and file size for images
- Header/Footer Manager: Add customizable headers/footers
- Watermarking Tool: Optional watermark addition

5. Data Flow

6. User Interaction:

- User authenticates → Submits URL with configuration
- Frontend validates input → Sends to backend API
- Job created and added to queue → Returns job ID to frontend

7. Processing Flow:

- Job scheduler picks up task → Initiates crawler
- Crawler respects robots.txt and rate limits
- Pages processed through extraction pipeline
- Content cleaned and organized → Stored in temporary storage
- Progress updates sent to frontend via WebSockets

8. PDF Generation:

- Processed content assembled into document structure
- Templates and styles applied → TOC and bookmarks generated
- Preview generated for user approval
- Final PDF created upon confirmation

9. Post-Processing:

- PDF stored in user's library → Download link generated
- Job status updated → Temporary files cleaned up
- Analytics data recorded

Technology Recommendations

- Frontend: Next.js, React, Shadcn UI, TailwindCSS
- Backend: Next.js API Routes, Vercel/Node.js serverless
- Queue: Bull MQ with Redis (for job management)
- Crawler: Puppeteer/Playwright for JS-heavy sites
- Content Extraction: Mozilla Readability or similar
- PDF Generation: React-PDF or PDFKit
- Storage: S3-compatible object storage
- Authentication: NextAuth.js for multiple auth providers
- Database: PostgreSQL for user data, job history, etc.
- Caching: Redis for temporary storage and performance

This architecture supports all the "must" requirements and most "should/may" requirements, with a clear path to add the remaining optional features.